

Implementing a hierarchical Bayseian model to determine enrichment of gene sets in
sequencing data

Abhi Thanvi, Kevin Maddox, & Simon Lizarazo Chaparro

University of Illinois at Urbana-Champaign

Author Note

The authors made the following contributions. Abhi Thanvi: Conceptualization, Writing - original draft preparation, Methodology; Kevin Maddox: Investigation, Project administration, Writing - original draft preparation; Simon Lizarazo Chaparro: Conceptualization, Data curation and formal analysis, Writing - review & editing.

Abstract

We propose and implement a hierarchical Bayesian model to assess gene set enrichment and differential expression, extending concepts learned in the classroom to a practical, real-world application. We tested our model on simulated data and conducted a brief sensitivity analysis to justify the chosen priors. Finally, we applied the model to real sequencing data from patients with Glioblastoma Multiforme (GBM) and their respective controls, comparing our results to those obtained using conventional methods. Model validation on synthetic data and subsequent Bayesian analysis on real-world data provide preliminary indications of the viability of this method, with limitations and proposed future work discussed briefly.

Implementing a hierarchical Bayesian model to determine enrichment of gene sets in sequencing data

Introduction

The development of genomic profiling techniques has significantly advanced the role of genetic biomarkers in the diagnosis and clinical management of cancer. These techniques offer potential predictive applications through single assays, enabling targeted treatments tailored to individual patients based on their genomic profiles. (Jimenez-Santos et al., 2022) Modern sequencing methods can generate a genetic profile of a sample by amplifying DNA or RNA fragments, allowing detection of the full spectrum of expressed genes without requiring full genome sequencing. Additionally, bioinformatic tools make it possible to identify genes that are differentially expressed between control and treatment or disease groups.

However, since observed biological effects often result from the activation of multiple gene signaling pathways and molecular mechanisms, analyzing individual gene expression alone is insufficient to explain complex biological processes. Beyond identifying single up- or down-regulated genes, it is crucial to understand the regulatory or biological pathways that drive or are impacted by these changes. Gene Set Enrichment Analysis (GSEA) (Mootha et al., 2003) is a widely used method to determine whether predefined sets of genes exhibit statistically significant differences in expression under different experimental conditions. GSEA ranks genes based on expression changes and calculates an enrichment score using a Kolmogorov-Smirnov-like test to evaluate the distribution of ranked genes within predefined sets. This score is then compared against a null distribution generated through permutation tests to assess significance.

Despite its utility, the nonparametric nature of GSEA and its reliance on empirical null distributions are recognized as limitations, and is a primary motivator of the method under study (Hukku et al., 2020).

Methods

Bayesian analysis was performed using the R programming language (R Core Team, 2022), the MCMC libraries `rjags` (Plummer et al., 2024b) and `coda` (Plummer et al., 2024a), and the JAGS software package (Plummer, 2017). Conventional GSEA was performed with the aid of the `DESeq2` package (M. Love, 2017).

Data

Two expression datasets were used in this analysis: one simulated dataset and one real dataset. The simulated dataset contained a total of 1,000 genes across 10 experiments, with 5 control and 5 treatment samples. Additionally, 50 gene sets were generated, each comprising 20 genes. For evaluating the effects at both the gene and gene set levels, we introduced specific treatment effects into the data as follows: all genes in gene set 1 were assigned an effect, while in gene sets 2, 3, and 4, only 10, 2, and 5 genes, respectively, were assigned an effect (**Figure 1B**).

The real dataset consisted of RNA expression data from glioblastoma multiforme (GBM) patients, including 5 patients and 5 controls. This data is publicly available and was retrieved from The Cancer Genome Atlas (TCGA) (Network, 2013). Both normalized and non-normalized data were downloaded. The normalized data, processed using the RPKM (Reads Per Kilobase of transcript per Million mapped reads) method for single-end data, was used for the Bayesian analysis. This normalization corrects for variations in library size and gene length. The non-normalized data was used for the conventional method (`DESeq2`), following the package’s user guidelines, which include a built-in normalization function.

Gene sets were obtained from the Human Molecular Signatures Database (MSigDB) (Submaranian et al., 2020), specifically the hallmark gene sets, which include 50 sets representing approximately 4,000 genes. Since the expression matrix contained a large number of genes (approx. 60,000), we subset the data to include only genes present in the

hallmark gene sets, along with 4,000 additional genes randomly sampled. To facilitate the analysis, we added 20 extra “gene sets” composed of these 4,000 randomly sampled genes. Each additional set contained 200 unique genes and was not expected to show enrichment. This subsetting was necessary to ensure the model could run efficiently.

Hierarchical Model

The i th observed expression value of gene g in set s is denoted as Y_{sgi} , which is assumed to follow a normal distribution with a mean μ_{sgi} and a precision τ_g^2 . The mean μ_{sgi} can be represented by a linear model that depends on α_{sg} and β_{sg} , where α_{sg} represents the overall mean expression values for gene g in set s and β_{sg} is the coefficient effect associated with the class label $X_i \in [0, 1]$, with 0 and 1 representing control and treatment classes, respectively. Therefore, the posterior distribution of β_{sg} can be used to determine differences between class variables at the gene level. We assume that β_{sg} follows a normal distribution with mean 0 and precision τ_s^2 as a hyperparameter. This hyperparameter is shared by all genes that belong to the gene set S_s and it adjusts the distribution of β_{sg} within a gene set, so that if a gene is not differentially expressed, then its posterior values for β_{sg} will move close to 0, corresponding to by large values of τ_s^2 , or small variance. Moreover, if a gene is significant in its posterior, there is a shift in its expression values between the two conditions. This will cause the posterior distribution β_{sg} to move away from 0. If a set has a large number of significant genes, then τ_s^2 becomes smaller in the posterior, allowing for large values of β_{sg}^2 . Gene set enrichment can be evaluated then according to the posterior distribution $\frac{1}{\tau_s^2}$: if the distribution shrinks close to 0, that gene set is not enriched. Nevertheless, large values represent enrichment for that set between those two conditions. Therefore, our hierarchical model is represented below.

Likelihood:

$$Y_{sgi} \sim \text{Normal}(\mu_{sgi}, \tau_g^2)$$

$$\mu_{sgi} = \alpha_{sg} + \beta_{sg} \times X_i$$

Priors:

$$\tau_g^2 \sim \text{InvGamma}(0.001, 0.001)$$

$$\alpha_{sg} \sim \text{Normal}(0, 0.000001)$$

$$\beta_{sg} \sim \text{Normal}(0, \tau_s^2)$$

Hyperprior:

$$\tau_g^2 \sim \text{InvGamma}(0.001, 0.001)$$

The parameters β_{sg} and τ_s^2) can then be used to determine the differential expression at the gene level and enrichment at the gene-set level. The inclusion of additional conditions were determined to complicate the model and were thus not explored further. The directed acyclic graph (DAG) for the proposed model is displayed in **Figure 1A**. The two variables of interest are highlighted; it is their posterior distributions that were collected during the analysis. In total there are a total of S parameters for τ_s^2) and G for β_{sg} .

Sensitivity Analysis

The proposed hierarchical model relies on uninformative priors, which we justify on the basis of each gene having distinct biological functions with unequal expression across different cells or tissues. This avoids the challenges associated with specifying different priors for the mean expression of every gene and for the effect associated with class variables. Furthermore, using a simpler model allows for analysis of multiple gene sets, which have a high degree of inherent heterogeneity, depending on the biological or cellular pathways of interest.

Considering that, we conducted a small sensitivity analysis to support this approach by implementing very informative priors for the following parameters: α_{sg} , τ_s^2 and τ_g^2 . This analysis was performed exclusively on the simulated dataset, as it was specifically designed to observe targeted effects.

$$\alpha_{sg} \sim \text{Normal}(0, 10)$$

$$\tau_s^2 \sim \text{InvGamma}(5, 5)$$

$$\tau_g^2 \sim \text{InvGamma}(0.001, 0.001)$$

Alternately, the exploration of altogether different prior distributions for assessment and analysis could be desirable but was not considered.

Conventional Method Analysis

To identify genes that were differentially expressed between the Control and Treatment groups, we employed DESeq2, a statistical method designed to analyze count-based NGS data, such as RNA-seq (Wang, Gerstein, & Snyder, 2009). DESeq2 models count data using a negative binomial distribution, adjusting for library size and gene-specific dispersion. The DESeq2 workflow involves the following key steps (M. I. Love, Huber, & Anders, 2014):

1. Data input and preprocessing
 - a. Raw read counts of gene expression were organized into a matrix format, with genes as rows and samples as columns.
 - b. Appropriate metadata (e.g., sample conditions, replicates) were included to help the model account for variability.
2. Normalization
 - a. DESeq2 uses the median-of-ratios method to normalize the data, addressing differences in sequencing depth across samples.
 - b. This normalization ensures that comparisons of expression levels are not biased by technical sequencing variations.
3. Model Fitting
 - a. Gene expression was modeled using a generalized linear model (GLM) based on the negative binomial distribution.

- b. Each gene's expression was regressed on the treatment condition to estimate the effect size attributable to the treatment.
4. Statistical Testing
 - a. Wald tests were performed to calculate p-values for each gene, testing the null hypothesis that there is no effect of the treatment condition on gene expression.
5. Adjustment for multiple testing
 - a. p-values were adjusted using the Benjamini-Hochberg procedure to control the false discovery rate (FDR), reducing the chance of false positives due to multiple comparisons.

Following the differential expression analysis, we used the Fast Gene Set Enrichment Analysis (FGSEA) method to test for enrichment of predefined gene sets, providing insight into the biological pathways impacted by the treatment. The FGSEA workflow includes the following steps:

1. Ranking of genes
 - a. Genes were ranked based on their differential expression statistics:
$$\text{Ranking Score} = \ln(\text{Fold Change}) \times \log_{10}(\text{p-value})$$
 - b. This ranking reflects both the magnitude and direction of changes.
2. Enrichment calculation
 - a. FGSEA calculates an enrichment score (ES) for each gene set, reflecting the extent to which genes from the set are overrepresented at the top or bottom of the ranked list.
 - b. This score is calculated by walking down the gene list, increasing a running-sum statistic when a gene in the set is encountered and decreasing it when it is not.
3. Normalization and statistical testing
 - a. Enrichment scores are normalized for gene set size to allow comparison across sets.
 - b. The analysis involves permuting the labels of the gene rankings and recalculating

the enrichment scores to build a null distribution.

4. Multiple testing correction

- a. Similar to DESeq2, FGSEA adjusts p-values to control the false discovery rate (FDR) using the Benjamini-Hochberg method.

Results

Bayesian analysis on simulated data

We ran MCMC simulation on the synthetic data in order to perform standard simulation monitoring as outlined in standard texts (Reich & Ghosh, 2019). Per recommended practice, we initialized parameters α_{sg} , β_{sg} , τ^2 with several different sets of starting values, choosing the three following: (0, 1, 1), (5, 5, 5) and (10, 10, 10). In **Figure 1C** we observe trace plots for two genes (out of 1000) two gene sets (out of 50), showing satisfactory sampling of the parameter space. Convergence was confirmed by monitoring Gelman plots for a single gene and gene set, shown in **Figure 1D**. Convergence is achieved and a total of 2000 iterations are burned before posterior analysis. The Gelman statistics for all gene sets in the simulated data are shown in **Figure 1E**, with most (but notably not all) genes presenting values below the standard threshold of 1.05, again indicating desired convergence behavior. As expected, the posterior distributions (**Figure 1E**) for β_{sg} show no overlap with 0 for gene set 1, in which all genes were assigned a treatment effect. Gene set 4 has a bimodal posterior, with some genes centered at 0 and some not. Since only some of its genes were assigned a treatment effect, this is also expected behavior. And finally, in gene set 10, where no genes were assigned treatment effect, the posterior is centered very close to 0. In **Figure 1F** the posterior distributions for σ_s^2 , which are anticipated to have values far from 0 for enriched gene sets (recall that sets 1, 2, and 3 were the enriched sets). and to be shrunk towards 0 for non-enriched sets. Finally, **Figure 1G**, summarizes the key finding of sensitivity analysis on the choice of parameters: the inclusion of very informative priors shifts the posterior distribution of σ_s^2 and β_{sg} further from 0, which can lead to inaccurate

biological interpretation of the results.

Bayesian analysis on GBM patients and their respective controls

Figure 2A shows boxplot examples for gene expression in controls and treatment across two different gene sets; random gene sets were generated by randomly sampling 200 genes not belonging to the original gene sets obtained from the MSigDB, and are not expected to be enriched in our analysis. Trace plot analysis (**Figure 2B**) shows proper mixing, as evidence by the ZMIZ1 gene and apoptosis gene set (a total of 8000 genes and 69 gene sets were considered). Example Gelman polts, in this case for the EIF2S1 gene and coagulation gene set show desired convergence behavior, with 2000 iterations burned as with simulated data, and estimated Gelman statistics for nearly all gene sets (**Figures 2C and 2D**) present below the 1.05 convergence threshold. The posterior means and 95% credible intervals for β_{sg} are displayed in **Figure 2E** for the top 20 genes in each category that were found to be considered as downregulated, upregulated, and without effect. Venn diagrams displaying the total overlap between upregulated and downregulated genes determined by the conventional method (DESeq2) and the hierarchical Bayesian model are displayed in **Figure 2F**. In **Figure 2G**, the posterior means and credible intervals for σ_s^2 are shown for the top 30 gene sets: as expected, none of the randomly generated gene sets had posterior distributions for this parameter that moved away from 0. By comparison, **Figure 2H** shows gene set enrichment scores for the same 30 gene sets as in **2G** obtained via the conventional method, with observations in green determined to be significant by this method, and those in grey, insignificant. Finally, **Table 1** and **Table 2** show posterior statistics for a subset of genes and gene sets that were analyzed.

Discussion

The analysis on simulated data was able to demonstrate mixing and convergence of the proposed model. Furthermore, with synthetic data we were able to verify that the analysis indeed identifies treatment effects where they were expected to be observed. This aligns with

the results obtained by the reference paper (Shahababa, 2011). It is important to mention that our model is not exactly the same as theirs, given that they consider a more complex model with random effects and additional hyperpriors. The optimal performance in the simulated data allowed us to explore the results in real datasets, and we were able to observe that our model performs optimally as well in the GBM dataset.

In the analysis for both datasets the fact that some parameters had a relatively high Gelman Point Estimate, can be due multiple reasons, such as poor mixing for those specific parameters that can derive from high correlations or not enough number of iterations for the analysis. An optimal way to circumvent this problem could include increasing the number of chains, burn-in period and also the total number of iterations, nevertheless due to the magnitude of the dataset, achieving this becomes computationally demanding. Implementation of this type of models in high throughput computing environments can help with larger datasets and sampling numbers.

The differences observed between DESeq2 and our Bayesian method relies in the analytical nature of each. DESeq2 fits a negative binomial distribution to the data and implements a normalization method that accounts for the spread and variance intrinsic to the count data. Moreover, compared to our analysis it considers all the genes. Our Bayesian model uses different distributions since it does not rely on count data, and due to the computational burden it is accounting only for a small set of genes. However, more than 50% of the genes identified as up- and downregulated by our Bayesian analysis were also identified by DESeq2.

On the other hand, FGSEA, as mentioned previously, is based on a ranked method and non parametric distributions for its analysis. A major limitation is that it ignores gene-gene correlations (Tamayo et al., 2016) that are accounted for by our Bayesian methods, which suggests it as a preferable method. Nevertheless, the gene set MYC_TARGETS appeared as the most likely enriched gene set in our Bayesian analysis and this specific gene set was also

observed in the FGSEA.

Limitations

The primary limitations of the particular method of analysis presented here, and our model, are the computational demand in comparison to conventional methods, and the fact that the proposed method does not allow analysis for multiple comparisons. The fact that genes can only be present in a single gene set is, in the scope of work conducted, a methodological necessity without clear biological motivation. Additionally, the simulated dataset was designed under specific conditions to observe pre-defined effects. While this approach allowed for controlled testing of the model, it may not fully capture the complexity and variability present in real biological data.

Another limitation, which we would have liked to explore further with more time, is the choice of priors in the hierarchical Bayesian model. While sensitivity analysis supported the use of uninformative priors, the selection of priors remains a critical aspect of Bayesian inference. Alternative priors could yield different results, and this dependency may impact the interpretation of enrichment scores and effect sizes.

In summary, these limitations cumulatively affect the biological interpretation and significance of our findings. Therefore, we find it necessary to point out these limitations in addition to our results. Some future work we would recommend includes exploring the effects of prior selections and working with more robust, complete datasets, alongside the necessary computational resources.

References

- Hukku, A. et al. (2020). BAGSE: A bayesian hierarchical model approach for gene set enrichment analysis. *Bioinformatics*, 36(6), 1689–1695.
<https://doi.org/10.1093/bioinformatics/btz831>
- Jimenez-Santos et al. (2022). Bioinformatics roadmap for therapy selection in cancer genomics. *Molecular Oncology*, 16(21), 3881–3908.
<https://doi.org/10.1002/1878-0261.13286>
- Love, M. (2017). *DESeq2: Differential gene expression analysis based on the negative binomial distribution*. Retrieved from
<https://rdocumentation.org/packages/DESeq2/versions/1.12.3>
- Love, M. I., Huber, W., & Anders, S. (2014). Moderated estimation of fold change and dispersion for RNA-seq data with DESeq2. *Genome Biology*, 15, 1–21.
<https://doi.org/10.1186/s13059-014-0550-8>
- Mootha, V. K. et al. (2003). PGC-1 α -responsive genes involved in oxidative phosphorylation are coordinately downregulated in human diabetes. *Nature Genetics*, 34, 267–273.
<https://doi.org/10.1038/ng1180>
- Network, T. C. G. A. R. (2013). The cancer genome atlas pan-cancer analysis project. *Nature Genetics*, 45, 1113–1120. <https://doi.org/10.1038/ng.2764>
- Plummer, M. (2017). *JAGS version 4.3.0 user manual*. Retrieved from
<https://sourceforge.net/projects/mcmc-jags>
- Plummer, M. et al. (2024a). *Coda: Output analysis and diagnostics fro MCMC*. Retrieved from <https://CRAN.R-project.org/package=coda>
- Plummer, M. et al. (2024b). *Rjags: Bayesian graphical models using MCMC*. Retrieved from <https://cran.r-project.org/package=rjags>
- R Core Team. (2022). *R: A language and environment for statistical computing*. Vienna, Austria: R Foundation for Statistical Computing. Retrieved from
<https://www.R-project.org>

- Reich, B. J., & Ghosh, S. K. (2019). *Bayesian statistical methods*. New York, NY: Chapman & Hall/CRC.
- Shahababa, B. et. al. (2011). Bayesian gene set analysis for identifying significant biological pathways. *Journal of the Royal Statistical Society Series C: Applied Statistics*, 60(4), 541–557. <https://doi.org/10.1111/j.1467-9876.2011.00765.x>
- Submaranian, A. et al. (2020). Gene set enrichment analysis: A knowledge-based approach for interpreting genome-wide expression profiles. *PNAS*, 102(43), 15545–15550. <https://doi.org/10.1073/pnas.0506580102s>
- Tamayo, P. et al. (2016). The limitations of simple gene set enrichment analysis assuming gene independence. *Statistical Methods in Medical Research*, 25(1), 472–487. <https://doi.org/10.1177/0962280212460441>
- Wang, Z., Gerstein, M., & Snyder, M. (2009). RNA-seq: A revolutionary tool for transcriptomics. *Nature Reviews Genetics*, 10, 57–63. <https://doi.org/10.1038/nrg2484>

Appendix

Table 1: Posterior statistics for a subset of genes analyzed

Genes	Mean	SD	Naive SE	2.50%	50%	97.50%	status
PABPC1	30.9673	5.9545	0.0384	16.1721	31.9390	39.7765	Upregulated
HNRNPA1	26.8730	9.2886	0.0600	5.6315	28.1725	41.5027	Upregulated
SET	24.3779	5.9927	0.0387	10.2540	25.1014	34.1989	Upregulated
PDIA6	24.1831	5.5768	0.0360	9.7516	25.4101	31.8191	Upregulated
HNRNPC	22.3616	8.4933	0.0548	3.2847	23.2161	36.7235	Upregulated
MCM7	21.8435	8.8100	0.0569	2.5948	22.6835	36.8203	Upregulated
HDGF	21.1815	9.4599	0.0611	1.2694	21.9609	37.5738	Upregulated
PPM1G	20.2199	4.0109	0.0259	11.1678	20.5346	27.2576	Upregulated
FBL	19.2760	6.8553	0.0443	4.0039	19.9024	31.0934	Upregulated
SERBP1	19.1990	6.3128	0.0407	5.2569	19.6971	30.2779	Upregulated
EIF3B	16.9313	3.3664	0.0217	9.6165	17.1600	23.0204	Upregulated
SNRPG	16.6760	6.7369	0.0435	2.0100	17.1816	28.5696	Upregulated
PSPH	16.6308	4.0535	0.0262	7.1265	17.1284	23.2238	Upregulated
EIF3D	16.6127	3.8884	0.0251	8.0630	16.8531	23.6690	Upregulated
ITGB2	16.5130	3.6430	0.0235	6.0965	17.3307	21.2366	Upregulated
DCTN1	-25.0510	8.8761	0.0573	-36.7369	-27.3831	-4.0131	Downregulated
BCL2L2	-23.9884	7.3321	0.0473	-33.4806	-25.9470	-4.7838	Downregulated
MADD	-16.9704	6.2909	0.0406	-27.0777	-17.8147	-2.6417	Downregulated
ATP6V0D1	-16.7243	3.4996	0.0226	-22.6067	-17.1179	-8.6284	Downregulated
KIF5C	-16.0417	5.3633	0.0346	-23.3008	-17.3926	-2.8723	Downregulated
ZNF365	-15.7888	3.9108	0.0252	-20.4604	-16.8646	-4.8062	Downregulated
FARSA	-14.9485	3.0729	0.0198	-20.6725	-15.1226	-8.3638	Downregulated
DNAJA4	-14.3843	3.6825	0.0238	-20.8345	-14.6998	-6.2526	Downregulated
NEFH	-14.1978	4.6153	0.0298	-22.1011	-14.6192	-3.7172	Downregulated
NCOA7	-13.2639	1.9616	0.0127	-16.6754	-13.4415	-8.8505	Downregulated
SLC1A4	-12.5112	3.9791	0.0257	-19.4591	-12.8579	-3.6454	Downregulated
ARRB1	-11.8448	3.0157	0.0195	-17.3560	-12.0161	-5.2770	Downregulated
PAFAH1B1	-10.7014	3.9563	0.0255	-17.1456	-11.1980	-1.8855	Downregulated
GABRA3	-10.1755	3.6986	0.0239	-16.4902	-10.5346	-1.9808	Downregulated
AIFM3	-9.6767	1.7380	0.0112	-13.0063	-9.7325	-6.0278	Downregulated
ACO2	-9.0945	2.2414	0.0145	-12.8569	-9.3361	-3.9159	Downregulated
MAP1S	-9.0904	1.8817	0.0121	-12.3119	-9.2632	-4.8843	Downregulated
NTM	-8.9512	2.5699	0.0166	-13.6096	-9.1147	-3.4221	Downregulated
ALDH6A1	-8.6833	1.9176	0.0124	-11.9661	-8.8583	-4.3845	Downregulated
BTBD3	-8.5993	3.8188	0.0247	-15.4541	-8.8685	-0.3592	Downregulated
MAGEE1	-11.9856	6.6113	0.0427	-24.2016	-12.1520	1.1401	No Effect
PPP3R1	-11.9781	9.6190	0.0621	-32.0855	-11.4415	5.4381	No Effect
PLPPR4	-11.5978	8.0938	0.0522	-27.4783	-11.5730	4.0583	No Effect
TNPO2	-10.2690	5.2837	0.0341	-19.7902	-10.5075	0.5577	No Effect
MAL2	-9.5479	5.4827	0.0354	-19.9168	-9.6061	1.1415	No Effect
RBP4	-9.4842	5.2043	0.0336	-19.0326	-9.6971	1.1860	No Effect
NLGN2	-9.2743	6.1648	0.0398	-21.4431	-8.9017	1.7366	No Effect
TSPYL2	-9.0052	8.7996	0.0568	-27.4254	-8.6126	7.4255	No Effect
BLCAP	-8.0948	3.8270	0.0247	-14.9105	-8.3243	0.0237	No Effect
EPB41L3	-8.0791	4.2248	0.0273	-15.6280	-8.3402	0.7410	No Effect
SERPINE2	-7.9479	3.8992	0.0252	-15.1418	-8.1413	0.3365	No Effect
MAPK11	-7.8566	4.7548	0.0307	-16.8852	-7.8643	1.3797	No Effect
CADM2	-7.8232	5.1164	0.0330	-17.9572	-7.6772	1.7595	No Effect
MMD	-7.7681	5.6174	0.0363	-18.7404	-7.7180	2.8935	No Effect

Table 2: Posterior statistics for gene sets

Sets	Mean	SD	Naive SE	2.50%	50%	97.50%
ADIPOGENESIS	15.5839	4.9609	0.0320	7.7859	14.9287	26.9581
ALLOGRAFT_REJECTION	5.7550	3.1968	0.0206	2.1738	4.8967	14.7407
ANDROGEN_RESPONSE	1.9724	1.8981	0.0123	0.0671	1.3631	7.0226
ANGIOGENESIS	1.4738	3.8581	0.0249	0.0004	0.0035	11.6907
APICAL_JUNCTION	23.2530	9.3753	0.0605	8.6671	22.1245	44.6018
APICAL_SURFACE	0.0467	0.0897	0.0006	0.0019	0.0163	0.2829
APOPTOSIS	72.0121	28.4724	0.1838	26.7794	68.6463	138.4447
BILE_ACID_METABOLISM	0.0014	0.0011	0.0000	0.0003	0.0011	0.0045
CHOLESTEROL_HOMEOSTASIS	31.7725	15.4042	0.0994	12.1376	28.5455	70.7840
COAGULATION	0.0007	0.0005	0.0000	0.0002	0.0005	0.0020
COMPLEMENT	6.3810	2.9856	0.0193	1.5066	6.1046	12.9700
DNA_REPAIR	10.9771	2.8294	0.0183	6.4873	10.6076	17.5211
EF_TARGETS	44.1166	7.9840	0.0515	30.5643	43.4089	61.6918
EPITHELIAL_MESENCHYMAL_TRANSITION	45.3679	16.0989	0.1039	21.1720	42.8778	83.5004
ESTROGEN_RESPONSE_EARLY	2.6397	1.2666	0.0082	0.7267	2.4415	5.6180
ESTROGEN_RESPONSE_LATE	1.6115	1.2784	0.0083	0.0351	1.3161	4.9826
FATTY_ACID_METABOLISM	0.5129	1.4908	0.0096	0.0003	0.0011	5.4132
GLYCOLYSIS	9.4014	3.5528	0.0229	4.4344	8.7492	18.3550
GM_CHECKPOINT	31.1726	7.9916	0.0516	18.2664	30.2501	49.5817
HEDGEHOG_SIGNALING	17.9863	30.5937	0.1975	0.0006	0.0268	99.8579
HEME_METABOLISM	5.8886	1.7900	0.0116	3.1927	5.6320	10.1164
HYPOXIA	7.1124	2.7388	0.0177	3.0152	6.6771	13.7459
IL_JAK_STAT_SIGNALING	16.6727	7.8582	0.0507	5.1004	15.4886	35.1588
IL_STAT_SIGNALING	3.2753	1.1077	0.0072	1.6044	3.1085	5.8668
INFLAMMATORY_RESPONSE	11.4692	3.8889	0.0251	4.0976	11.2328	19.8528
INTERFERON_ALPHA_RESPONSE	38.6626	11.9866	0.0774	21.1713	36.6908	67.7855
INTERFERON_GAMMA_RESPONSE	13.3065	3.2937	0.0213	7.9661	12.9324	20.7956
KRAS_SIGNALING_DN	0.0012	0.0009	0.0000	0.0002	0.0009	0.0037
KRAS_SIGNALING_UP	29.8169	8.1684	0.0527	13.6946	29.5593	46.7829
MITOTIC_SPINDLE	19.5624	4.1365	0.0267	12.6201	19.1608	28.6548
MTORC_SIGNALING	52.6885	13.1491	0.0849	31.3373	51.1128	82.7549
MYC_TARGETS_V	120.5063	23.8797	0.1541	79.4726	118.5820	173.3860
MYOGENESIS	0.0012	0.0011	0.0000	0.0003	0.0009	0.0039
NOTCH_SIGNALING	72.4854	33.3846	0.2155	29.1586	65.4613	155.4530
OXIDATIVE_PHOSPHORYLATION	20.2925	5.6753	0.0366	11.2317	19.6093	33.2811
P_PATHWAY	21.3938	5.6820	0.0367	11.5689	20.9165	34.0673
PANCREAS_BETA_CELLS	0.0019	0.0040	0.0000	0.0003	0.0011	0.0085
PEROXISOME	5.8292	3.9455	0.0255	1.3703	4.7356	16.0838
PIK_AKT_MTOR_SIGNALING	14.9497	6.2399	0.0403	6.2792	13.7877	30.3915
PROTEIN_SECRETION	26.6635	8.4671	0.0547	13.7275	25.4826	46.8383
REACTIVE_OXYGEN_SPECIES_PATHWAY	20.9664	18.0337	0.1164	3.1201	15.8361	68.9607
SPERMATOGENESIS	0.0006	0.0004	0.0000	0.0002	0.0005	0.0017
TGF_BETA_SIGNALING	13.6303	5.5241	0.0357	6.0668	12.5758	27.3230
TNFA_SIGNALING_VIA_NFKB	29.5792	8.2789	0.0534	16.6887	28.4528	48.9896
UNFOLDED_PROTEIN_RESPONSE	66.7513	19.4206	0.1254	34.0750	64.9811	109.9594
UV_RESPONSE_DN	10.8089	3.5152	0.0227	5.4244	10.3043	18.9554
UV_RESPONSE_UP	11.8559	5.1857	0.0335	5.1318	10.7523	25.1380
WNT_BETA_CATENIN_SIGNALING	15.5426	6.9547	0.0449	5.0572	14.5156	31.9532
XENOBIOTIC_METABOLISM	0.0005	0.0003	0.0000	0.0002	0.0004	0.0013

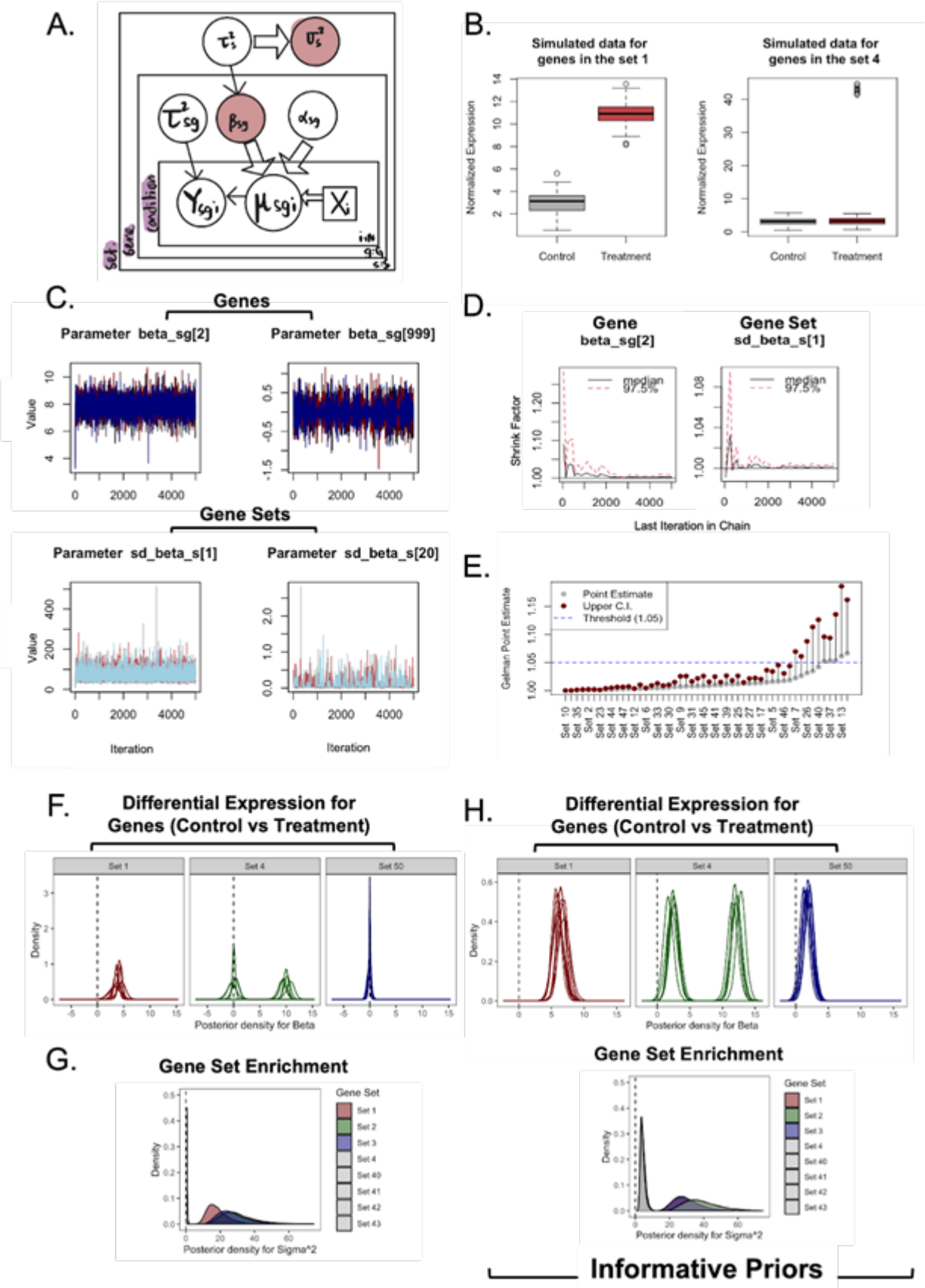


Figure 1. Bayesian analysis on simulated data

Figure 1. Bayesian analysis on simulated data. **A.** Directed Acyclic Graph (DAG) for the proposed model. Three different plates are considered, one for each condition (control or treatment), one for each gene, and for each gene set. Highlighted are the parameters with posteriors of interest. **B.** Boxplots examples for gene expression in controls and treatment across two different gene sets. For gene set 1, intentionally all the genes have a higher normalized expression in treatment vs control. For gene set 4 only 5 out of 20 genes have a higher normalized expression in treatments vs control. **C.** Trace plot for two genes (out of 1000) and two gene sets (out of 50). Mixing is observed. **D.** German plots for a single gene and a single gene set. Convergence is achieved and a total of 2000 iterations are burned prior to posterior analysis. **E.** Gelman point estimates for all the gene sets. The plot represents the estimate and the upper credible interval for all the genes along with a dotted blue line that represents the threshold. Most of the gene sets presented values under the threshold. **F.** Posterior distributions for parameter Beta_sg . Distributions are displayed for genes belonging to three different sets. For set 1 all the genes appear to have a posterior that does not overlap 0; for set 4 some genes are far from 0 while others are centered on 0; for set 50 which is not supposed to have any gene effect between control and treatments show a sharp distribution around 0 for all the genes. **G.** Posterior distributions for sigma_s . Gene sets that are enriched will have values far from 0, while gene sets that are not enriched will have a posterior for sigma_s that shrinks towards 0. **H.** Sensitivity analysis on posterior distributions for genes and gene sets displayed previously. The inclusion of very informative priors shifts parameter distributions for from. Genes and gene sets that were simulated to not be enriched appear as such with this type of priors.

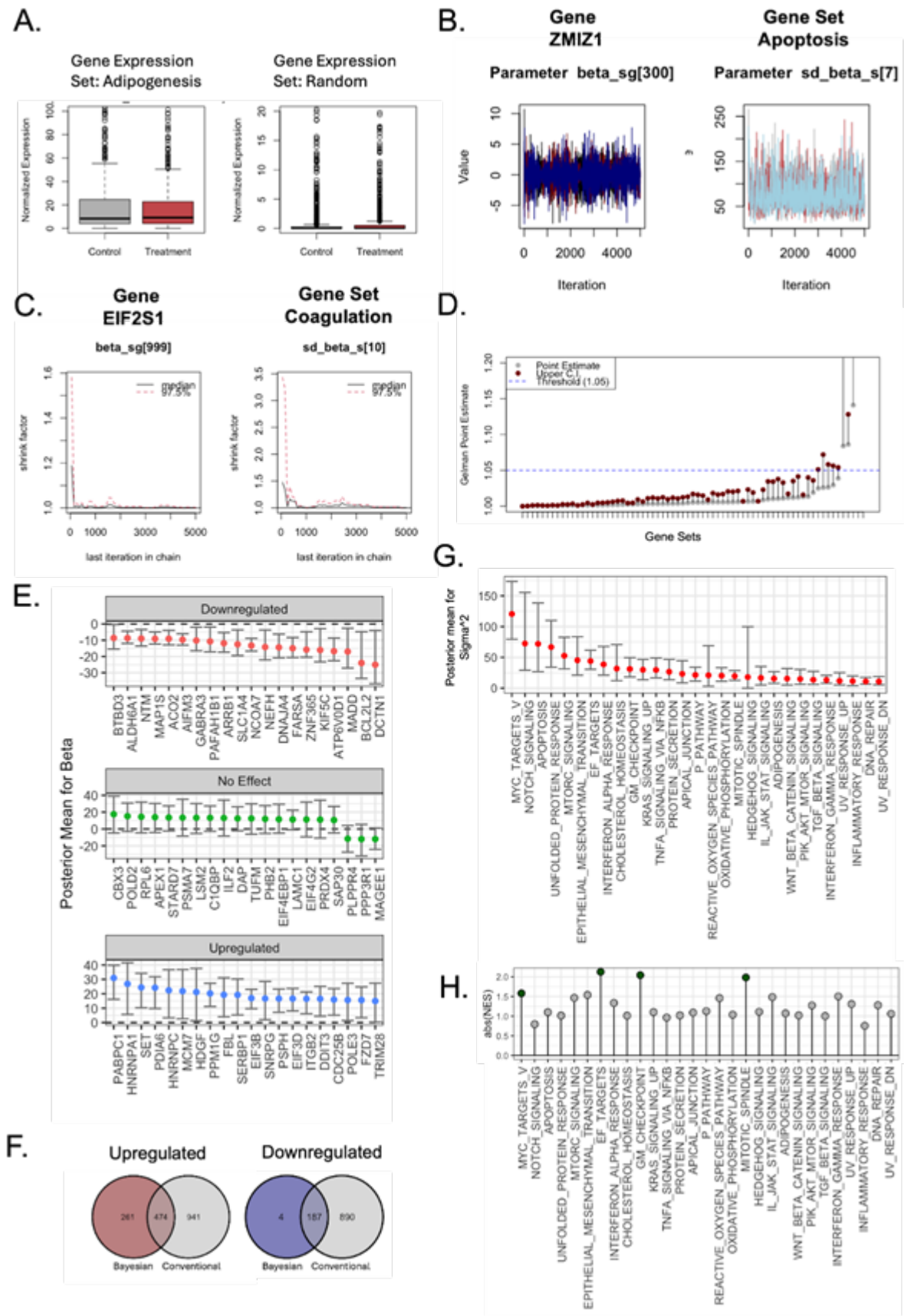


Figure 2. Bayesian analysis on patients with GBM and their respective controls

Figure 2. Bayesian analysis on patients with GBM and their respective controls. **A.** Boxplots examples for gene expression in controls and treatment across two different gene sets. Random sets are generated by sampling 200 genes that do not belong to the original gene sets obtained from the Molecular Signatures Database (MSigDB). These random sets are not expected to be enriched in our analysis. **B.** Trace plot for the ZMIZ1 gene and the gene set apoptosis. A total of ~8000 genes were considered along with 69 gene sets. **C.** Gelman plot for the EIF2S1 gene and the gene set coagulation. A total of 2000 Iterations were burned prior to posterior inference. **D.** Gelman point estimates for all the gene sets. The plot represents the estimate and the upper confidence interval for all the genes along with a dotted blue line that represents the threshold. Most of the gene sets presented values under the threshold. **E.** Posterior means and credible intervals for genes considered as Downregulated, Upregulated, and without an effect. Only the top 20 for each category are being represented. **F.** Venn diagrams displaying the total overlap between upregulated and downregulated genes determined by the conventional method (DESEQ2) and our Bayesian model. **G.** Posterior means and confidence intervals for the top 30 gene sets. None of the random gene sets had posterior distributions for sigma that moved away from 0. **H.** Scores for gene set enrichment analysis performed using the conventional method (FGSEA). Gene sets represented are the same one as the ones that appear in **G**. Observations in green are called significant by the method and those in gray are non-significant.

Implementing a hierarchical Bayseian model to determine enrichment of gene sets in sequencing data - Appendix

Simon Lizarazo Kevin Maddox Abhi Thanvi

2024-12-08

```
knitr::opts_chunk$set(echo = TRUE)
library(tidyverse)
library(rjags)
library(tidyverse)
library(DESeq2)

library(ggvenn)
library(fgsea)
library(rjags)
library(TCGAbiolinks)
library(GenomicDataCommons)
```

Simulated Data

This markdown has information regarding the analysis for gene set enrichment on simulated data.

The simulated data is a total of 1000 genes G in 10 experiments N , out of which 5 are control and 5 are treatment. There are a total of 50 gene sets `num_sets`, and each set has 20 genes `genes_per_set`. These genes are unique to each gene, meaning that a single gene cannot be present in another set.

```
N <- 10      # Total samples
G <- 1000     # Total genes
num_sets <- 50 # Total gene sets
genes_per_set <- 20 # Genes per set

# Gene sets (assign each gene to a set)
gene_sets <- rep(1:num_sets, each=genes_per_set)
names(gene_sets) <- paste0("Gene", 1:G)

# Sample indices for control and treatment groups
control_indices <- 1:5
treatment_indices <- 6:10

# Class variable (0: control, 1: treatment)
x <- c(rep(0, 5), rep(1, 5))

Y <- matrix(rnorm(G*N, mean=3, sd=1), nrow=G, ncol=N)
rownames(Y) <- paste0("Gene", 1:G)

Y[1:10,1:10] |> as.data.frame()
```

The generated data, assumes a normalized matrix for gene expression across different conditions.

Now we create a function to add an effect to significant genes in the treatment group. Basically we are forcing some genes to have a significant difference in their normalized values in the treatment group vs control group

```
add_effect <- function(Y, genes, effect_size) {
  Y[genes, treatment_indices] <- Y[genes, treatment_indices] + effect_size
  return(Y)
}
```

We are adding this effect to different genes across different sets.

```
# Set 1: All 20 genes significant
Y <- add_effect(Y, genes=1:20, effect_size=4)

# Set 2: 10 significant genes (genes 21-30)
Y <- add_effect(Y, genes=21:30, effect_size=8)

# Set 3: 5 significant genes (genes 41-45)
Y <- add_effect(Y, genes=41:45, effect_size=10)

# Set 4: 2 significant genes (genes 61-62)
Y <- add_effect(Y, genes=61:62, effect_size=20)

## let's see an example

par(mfrow = c(1,2))

data = data.frame(expression = as.vector(Y[1:20, 1:10]), type = rep(c('Control', 'Treatment'), each = 10))

boxplot(data$expression ~ data$type,
        xlab = '', ylab = 'Normalized Expression',
        main = 'Simulated data for \n genes in the set 1', col = c('gray', 'indianred'))

data = data.frame(expression = as.vector(Y[61:80, 1:10]), type = rep(c('Control', 'Treatment'), each = 10))

boxplot(data$expression ~ data$type,
        xlab = '', ylab = 'Normalized Expression',
        main = 'Simulated data for \n genes in the set 4',
        col = c('gray', 'indianred'))
```

Our model should be able to determine that for each of these genes, there is a difference in the expression levels.

```
model_string <- "
model {
  for (i in 1:N) {
    for (g in 1:G) {
      # Likelihood
      Y[g, i] ~ dnorm(mu[g, i], tau_sg[g])
      mu[g, i] <- alpha_sg[g] + beta_sg[g] * x[i]
    }
  }

  for (g in 1:G) {
    # Priors for intercepts and slopes
```

```

    alpha_sg[g] ~ dnorm(0, 1e-6) # Non-informative prior for alpha
    beta_sg[g] ~ dnorm(0, tau_beta_s[gene_set[g]]) # Hierarchical prior for beta
    tau_sg[g] ~ dgamma(0.001, 0.001) # Prior for precision (1 / sigma^2)
  }

  for (s in 1:num_sets) {
    # Hyperpriors for the precision of beta (tau_beta_s)
    tau_beta_s[s] ~ dgamma(0.001, 0.001)
    sd_beta_s[s] <- 1 / (tau_beta_s[s]) # variance for interpretation
  }
}
"

writeLines(model_string, con="model.txt")

```

Hyierarchical Model Setting up the list for rjags

```

data_jags <- list(
  Y = Y,
  x = x,
  gene_set = as.numeric(gene_sets),
  N = N,
  G = G,
  num_sets = num_sets
)

```

Some initialization values. We decide to initialize the hyperparameters that do not receive input from another parameter.

```

inits_list <- list(
  list(
    alpha_sg = rep(0, G),
    tau_sg = rep(1, G),
    tau_beta_s = rep(1, num_sets)
  ),
  list(
    alpha_sg = rnorm(G, 5, 0.1),
    tau_sg = rep(5, G),
    tau_beta_s = rep(5, num_sets)
  ),
  list(
    alpha_sg = rnorm(G, 10, 0.1),
    tau_sg = rep(10, G),
    tau_beta_s = rep(10, num_sets)
  )
)

```

Parameters we are interested in. Sd_beta_s gives information about gene sets variance and beta_sg about gene effect given a condition (control or treatment)

```

# Parameters to monitor
parameters <- c("sd_beta_s", "beta_sg")

```

```

jags_model <- jags.model(file="model.txt", data=data_jags, n.chains=3, n.adapt=1000, inits = inits_list)

```

Due to the number of parameters we perform 5000 iterations for computational efficiency

```
samples <- coda.samples(model=jags_model, variable.names=parameters, n.iter=5000)
```

Let's assess mixing and convergence. We are showing an example for a single gene and single gene set.

```
# Convert samples to a matrix for easier manipulation
samples_matrix <- as.matrix(samples)

### mixing for 5 random genes

mixing_plot <- function(chains = 1:3, parameter = 1, cols = 1:3, model = samples) {
  # This will now collect the data for the specified parameter across the specified chains.
  data_to_plot <- sapply(chains, function(i) model[[i]][, parameter])

  # Since `parameter` is an index, use it directly to get the name outside of sapply
  name = colnames(model[[1]])[parameter]

  # Plot using matplot
  return(matplot(data_to_plot, type = 'l', lty = 1,
    xlab = "Iteration", ylab = "Value",
    main = paste("Parameter ", name),
    col = cols)) # Assign different colors for each chain
}

# Setting graphical parameters to plot multiple plots on one graphical device
par(mfrow = c(2,2))

# Example call to the function to plot the second parameter
mixing_plot(parameter = 2, cols = c('black', 'darkred', 'darkblue'))
mixing_plot(parameter = 999, cols = c('black', 'darkred', 'darkblue'))
mixing_plot(parameter = 1001, cols = c('gray', 'indianred', 'lightblue'))
mixing_plot(parameter = 1020, cols = c('gray', 'indianred', 'lightblue'))
```

Gelman plots

```
parameters_of_interest = c('beta_sg[2]', 'sd_beta_s[1]')

selected_samples <- lapply(samples, function(chain) {
  chain[, parameters_of_interest, drop = FALSE]
})

selected_samples <- as.mcmc.list(selected_samples)

gelman.plot(selected_samples)
```

Gelman diagnostic for all sets. Some sets do not meet the criteria, but most of them are under the threshold for convergence.

Based on the plots we decide to burn the first 2000 observations

```
diagnostic = gelman.diag(samples)
diagnostic = as.data.frame(diagnostic$psrf[1001:1050,])
diagnostic$set = paste('Set ', 1:50)

# Sorting the dataframe by 'Point est.'
diagnostic <- diagnostic[order(diagnostic$`Point est.`),]
```



```

# Update the factor levels after sorting
diagnostic$set <- factor(diagnostic$set, levels = unique(diagnostic$set))

# Setting up the plot area
plot(diagnostic$set, diagnostic$`Point est.` , type = 'n', # 'n' for no plotting
      xaxt = 'n',
      xlab = '', ylab = 'Gelman Point Estimate', pch = 19,
      ylim = range(c(diagnostic$`Point est.` , diagnostic$`Upper C.I.` , 1.1)))

# Adding points for the point estimates
points(diagnostic$set, diagnostic$`Point est.` , pch = 19, col = "gray", cex = 1)

# Adding points for the Upper Confidence Interval
points(diagnostic$set, diagnostic$`Upper C.I.` , pch = 19, col = "darkred", cex = 1)

# Adding lollipop sticks - segments from point estimates to upper CI
segments(as.numeric(diagnostic$set), diagnostic$`Point est.` ,
         as.numeric(diagnostic$set), diagnostic$`Upper C.I.` , col = 'black')

# Adding a dashed line at y = 1.05
abline(h = 1.05, col = "blue", lty = "dashed")

# Custom x-axis with labels rotated 90 degrees
axis(1, at = 1:length(levels(diagnostic$set)), labels = levels(diagnostic$set), las = 2)

# Adding a legend to the plot
legend("topleft", legend = c("Point Estimate", "Upper C.I.", "Threshold (1.05)"),
      col = c("gray", "darkred", "blue"), pch = c(19, 19, NA), lty = c(NA, NA, "dashed"))

```

Genes differential expression between control and treatment. Let's burn the first 2000 iterations.

```

jags_model <- jags.model(file="model.txt", data=data_jags, n.chains=3, n.adapt=1000, inits = inits_list)

samples <- coda.samples(model=jags_model, variable.names=parameters, n.iter=10000)

runs = window(samples, 2001)

runs1 = as.matrix(runs)

genes_of_i = runs1[,c(1:10, 41:50, 981:990)]

genes_of_i = genes_of_i |> as.data.frame() |> pivot_longer(cols = 1:30)

group_df = data.frame(genes = unique(genes_of_i$name))
group_df$sets <- rep(paste('Set', c(1,4,50)), each = 10)

genes_of_i = genes_of_i |> left_join(group_df, join_by('name' == 'genes'))

genes_of_i |> group_by(name)|> ggplot() +
  geom_density(aes(x = value, col = sets, group = name), show.legend = F) +
  facet_wrap(~sets) +
  theme_bw() +
  geom_vline(xintercept = 0, linetype = 'dashed') +

```

```
theme(aspect.ratio = 1, panel.grid = element_blank()) +
scale_color_manual(values = c('darkred', 'darkgreen', 'darkblue')) +
xlab('Posterior density for Beta') + ylab('Density')
```

Gene sets

```
sets_of_i = runs1[,c(1001:1004, 1040:1043)]

colnames(sets_of_i) = c(paste0('Set ', c(1:4, 40:43)))

sets_of_i = sets_of_i |> as.data.frame() |> pivot_longer(cols = 1:8)

sets_of_i %>%
  group_by(name) %>%
  ggplot(aes(x = value)) +
    geom_density(aes(y = ..density.., fill = name), alpha = 0.5, show.legend = TRUE) + # Ensure density
    geom_vline(xintercept = 0, linetype = 'dashed') +
    scale_y_continuous(limits = c(0, 0.5)) + # Optionally set common Y limits if needed
    scale_fill_manual(values = c('darkred', 'darkgreen', 'darkblue', rep('gray', 5))) +
    theme_bw() +
    theme(aspect.ratio = 1, panel.grid = element_blank()) +
    xlab('Posterior density for Sigma^2') +
    ylab('Density') + xlim(0,75) + guides(fill = guide_legend('Gene Set'))
```

Informative priors How would our analysis be influenced by the presence of very informative priors?. In the previous analysis, we intended to use uninformative hyperpriors and priors. Now let's use informative hyperpriors

```
model_string <- "
model {
  for (i in 1:N) {
    for (g in 1:G) {
      # Likelihood
      Y[g, i] ~ dnorm(mu[g, i], tau_sg[g])
      mu[g, i] <- alpha_sg[g] + beta_sg[g] * x[i]
    }
  }

  for (g in 1:G) {
    # Priors for intercepts and slopes
    alpha_sg[g] ~ dnorm(0, 10) # Non-informative prior for alpha
    beta_sg[g] ~ dnorm(0, tau_beta_s[gene_set[g]]) # Hierarchical prior for beta
    tau_sg[g] ~ dgamma(5, 5) # Prior for precision (1 / sigma^2)
  }

  for (s in 1:num_sets) {
    # Hyperpriors for the precision of beta (tau_beta_s)
    tau_beta_s[s] ~ dgamma(5, 5)
    sd_beta_s[s] <- 1 / (tau_beta_s[s]) # variance for interpretation
  }
}
"

writeLines(model_string, con="model_informative.txt")
```

We can observe how informative priors shift the posterior distribution of genes and gene sets that are meant to be close to 0 to the right. Presence of informative priors can influence our conclusions in the wrong way.

```
parameters <- c("sd_beta_s", "beta_sg")
jags_model <- jags.model(file="model_informative.txt", data=data_jags, n.chains=3, n.adapt=1000, inits =

samples <- coda.samples(model=jags_model, variable.names=parameters, n.iter=10000)

runs = window(samples, 2001)

runs1 = as.matrix(runs)

genes_of_i = runs1[,c(1:10, 41:50, 981:990)]

genes_of_i = genes_of_i |> as.data.frame() |> pivot_longer(cols = 1:30)

group_df = data.frame(genes = unique(genes_of_i$name))
group_df$sets <- rep(paste('Set', c(1,4,50)), each = 10)

genes_of_i = genes_of_i |> left_join(group_df, join_by('name' == 'genes'))

genes_of_i |> group_by(name)|> ggplot() +
  geom_density(aes(x = value, col = sets, group = name), show.legend = F) +
  facet_wrap(~sets) +
  theme_bw() +
  geom_vline(xintercept = 0, linetype = 'dashed') +
  theme(aspect.ratio = 1, panel.grid = element_blank()) +
  scale_color_manual(values = c('darkred', 'darkgreen', 'darkblue')) +
  xlab('Posterior density for Beta') + ylab('Density')

sets_of_i = runs1[,c(1001:1004, 1040:1043)]

colnames(sets_of_i) = c(paste0('Set ', c(1:4, 40:43)))

sets_of_i = sets_of_i |> as.data.frame() |> pivot_longer(cols = 1:8)

sets_of_i %>%
  group_by(name) %>%
  ggplot(aes(x = value)) +
    geom_density(aes(y = ..density.., fill = name), alpha = 0.5, show.legend = TRUE) + # Ensure density
    geom_vline(xintercept = 0, linetype = 'dashed') +
    scale_y_continuous(limits = c(0, 0.5)) + # Optionally set common Y limits if needed
    scale_fill_manual(values = c('darkred', 'darkgreen', 'darkblue', rep('gray', 5))) +
    theme_bw() +
    theme(aspect.ratio = 1, panel.grid = element_blank()) +
    xlab('Posterior density for Sigma^2') +
    ylab('Density') + xlim(0,75) + guides(fill = guide_legend('Gene Set'))
```

Real dataset

Data retrieval can be achieved by using the following code. We are subsetting for 5 samples and 5 controls for patients with and without GBM

Two datasets are downloaded from the same source. One without normalized counts which is used by the

conventional method (Package DESEQ and FGSEA in R) and another one with normalized counts that will be used by our bayesian analysis. Normalization method is RPKM (reads per kilobase of transcript per million reads mapped) for single-end data correct for variations in library size and gene length

```
### Query for data retrieval

### Project - Par1 Data Retrieval

### libraries
library(tidyverse)
library(DESeq2)

if (!require("BiocManager", quietly = TRUE))
  install.packages("BiocManager")

BiocManager::install("fgsea")

library(fgsea)
library(rjags)
library(TCGAbiolinks)
library(GenomicDataCommons)

setwd('~/.Dropbox/UIUC/Coursework/2024/STAT 431/Project/')

### Query
### Data is publically available at the TCGA website.
### We are performing a query to filter for the data that we are interested in by using the TCGAbiolinks

projects <- getGDCprojects()
a = getProjectSummary('TCGA-GBM')

query = GDCquery(project = 'TCGA-GBM',
                 data.category = 'Transcriptome Profiling')

output_query = getResults(query)
output_query = output_query |> filter(access == 'open' & experimental_strategy == 'RNA-Seq' & analysis_
output_query = output_query |> filter(sample_type %in% c('Primary Tumor', 'Solid Tissue Normal'))
output_query = output_query |> filter(grepl('-06-', output_query$cases))
output_query = output_query |> group_by(sample_type) |> top_n(5)

### These are the features we are interested in. We will download a total of 10 dataframes each one with
query_fin = GDCquery(project = 'TCGA-GBM',
                    data.category = 'Transcriptome Profiling',
                    experimental_strategy = 'RNA-Seq',
                    workflow.type = 'STAR - Counts',
                    access = 'open',
                    barcode = output_query$cases
                    )

a = getResults(query_fin)

### Download
GDCdownload(query_fin)
```

```

### Extracting counts
gbm_data = GDCprepare(query_fin, summarizedExperiment = T)
gbm_matrix = assay(gbm_data, 'unstranded')
gbm_matrix_norm = assay(gbm_data, 'fpkm_unstrand')

tamp = read.table('GDCdata/TCGA-GBM/Transcriptome_Profiling/Gene_Expression_Quantification/2ada884e-183')
tamp = tamp[-1:-5,]

gbm_matrix = as.data.frame(gbm_matrix)
gbm_matrix$geneid = tamp$V2

gbm_matrix = gbm_matrix |> select(11, 2,4,7:9, 1, 3,5,6, 10)
colnames(gbm_matrix)[2:11] = c(paste('GBM_Control', 1:5, sep='_'), paste('GBM_Tumor', 1:5, sep='_'))

gbm_matrix_norm = as.data.frame(gbm_matrix_norm)
gbm_matrix_norm$geneid = tamp$V2
gbm_matrix_norm = gbm_matrix_norm |> select(11, 2,4,7:9, 1, 3,5,6, 10)
colnames(gbm_matrix_norm) = colnames(gbm_matrix)

write.table(gbm_matrix, 'GBM.Glioblastoma.RNASEQ.TCGA.txt', sep=',')
write.table(gbm_matrix_norm, 'GBM.Glioblastoma.RNASEQ.TCGA_NORMALIZED.txt', sep=',')

```

Once we have obtained our dataframe we can do some adjustments

Given that the total number of genes is quite big, we are subsetting for the genes that are only present in the list of sets + other 4000 genes different to the ones present in the list randomly sampled. We are including 20 extra 'gene sets' with this randomly sampled genes. This is done to be able to run the model effectively. Each extra set has a total of 200 unique genes and they are not expected to be enriched, meaning the posterior variance for the gene sets is expected to shrink towards zero.

```

#### Differential Analysis
gbm_matrix = read.delim('GBM.Glioblastoma.RNASEQ.TCGA.txt', sep=',')
gbm_matrix_norm = read.delim('GBM.Glioblastoma.RNASEQ.TCGA_NORMALIZED.txt', sep=',')

aa = grepl('_PAR_Y', rownames(gbm_matrix))
gbm_matrix = gbm_matrix[!aa, ]

gbm_matrix_norm = gbm_matrix_norm[!aa,]

### These genes are mislabeled
replace = data.frame(set = c('EEIG1', 'G6PC1', 'H2AC25', 'HYCC2', 'MIX23', 'MYL11', 'NHERF1', 'NHERF4',
matrix = c('FAM102A', 'G6PC', 'H2AW', 'FAM126B', 'CCDC58', 'MYLPF', 'SLC9A3R1', 'PDZD3', 'MPP5', 'PHB',

gbm_matrix = gbm_matrix |>
  left_join(replace, by = c('geneid' = 'matrix')) |>
  mutate(geneid = ifelse(is.na(set), geneid, set))

gbm_matrix_norm = gbm_matrix_norm |>
  left_join(replace, by = c('geneid' = 'matrix')) |>
  mutate(geneid = ifelse(is.na(set), geneid, set))

duplicated = table(gbm_matrix$geneid) |> as.data.frame()
duplicated = duplicated |> filter(Freq > 1)

```

```

gbm_matrix = gbm_matrix |> filter(!geneid %in% duplicated$Var1 )

gbm_matrix_norm = gbm_matrix_norm |> filter(!geneid %in% duplicated$Var1 )

count.matrix = gbm_matrix[,c(-1, -12)]

rownames(count.matrix) = gbm_matrix$geneid

### gene sets
### gmt files can be found at gsea mbiol website
sets = gmtPathways('hallmark.symbols.gmt')

##3 unlisting and removing duplicated genes.
sets_unlist = sets |> unlist() |> as.data.frame()
sets_unlist$set = rownames(sets_unlist)

sets_unlist$set = gsub('\\d+', '', sets_unlist$set)
colnames(sets_unlist) = c('genes', 'set')

sets_unlist = sets_unlist |> group_by(genes) |>
  sample_n(size = 1) |> ungroup()

sets_unlist = sets_unlist |> filter(genes != 'CD3G')

#### genes to sample
genes_to_sample = gbm_matrix |> filter(!geneid %in% sets_unlist$genes)
genes_to_sample = genes_to_sample[sample(1:nrow(genes_to_sample), size = 4000, replace = F),]

genes_to_sample_sets = genes_to_sample |> select(geneid, set)
genes_to_sample_sets$set = paste0('RANDOM_SET ', sample(1:20))

colnames(genes_to_sample_sets)[1] = 'genes'

sets_unlist = rbind(sets_unlist, genes_to_sample_sets)

list_from_df = split(sets_unlist$genes, sets_unlist$set)

```

Data Manipulation for bayesian analysis

And index for each set and each gene is created

```

sets_unlist = sets_unlist[order(sets_unlist$set),]
sets_unlist$index_gene = paste('Gene', 1:nrow(sets_unlist))
sets_unlist$index_set = as.numeric(as.factor(sets_unlist$set))

### filtering for only those genes we will analyze in the normalized data.
gbm_matrix_norm1 = gbm_matrix_norm |> filter(geneid %in% sets_unlist$genes)

```

```
gbm_matrix_norm1 = gbm_matrix_norm1 |> left_join(sets_unlist[,c(1,3)], join_by('geneid' == 'genes') )

rownames(gbm_matrix_norm1) = gbm_matrix_norm1$index_gene

gbm_matrix_norm1 = gbm_matrix_norm1[sets_unlist$index_gene,2:11]
write.table(gbm_matrix_norm1, 'GBM.Glioblastoma.RNASEQ.TCGA.NORMALIZED.FILTERED.txt', sep = ',')
```

Setting up JAGS Data list

```
N1 <- 10      # Total samples
G1 <- nrow(gbm_matrix_norm1)    # Total genes
num_sets1 <- length(unique(sets_unlist$index_set)) # Total gene sets

# Gene sets (assign each gene to a set)
gene_sets <- sets_unlist$index_set
names(gene_sets) <- sets_unlist$index_gene

# Sample indices for control and treatment groups
control_indices <- 1:5
treatment_indices <- 6:10

# Class variable (0: control, 1: treatment)
x <- c(rep(0, 5), rep(1, 5))

Y <- gbm_matrix_norm1 |> as.matrix()

Y[1:10,1:10] |> as.data.frame()
```

Normalized Expression Example

```
## let's see an example

par(mfrow = c(1,2))

data = data.frame(expression = as.vector(Y[1:133, 1:10]), type = rep(c('Control', 'Treatment'), each = 5))

boxplot(data$expression ~ data$type,
        xlab = '', ylab = 'Normalized Expression', ylim = c(0,100),
        main = 'Simulated data for \n genes in the set 1 \n (HALLMARK_ADIPOGENESIS)', col = c('gray', 'gray'))

data = data.frame(expression = as.vector(Y[8184:8383, 1:10]), type = rep(c('Control', 'Treatment'), each = 5))

boxplot(data$expression ~ data$type,
        xlab = '', ylab = 'Normalized Expression', ylim = c(0,20),
        main = 'Simulated data for \n genes in the random set',
        col = c('gray', 'indianred'))
```

```
data_jags1 <- list(
  Y = Y,
  x = x,
  gene_set = as.numeric(gene_sets),
  N = N1,
  G = G1,
  num_sets = num_sets1
```

```
)
```

Initialization

```
inits_list <- list(
  list(
    alpha_sg = rep(0, G1),
    tau_sg = rep(1, G1),
    tau_beta_s = rep(1, num_sets1)
  ),
  list(
    alpha_sg = rnorm(G1, 5, 0.1),
    tau_sg = rep(5, G1),
    tau_beta_s = rep(5, num_sets1)
  ),
  list(
    alpha_sg = rnorm(G1, 10, 0.1),
    tau_sg = rep(10, G1),
    tau_beta_s = rep(10, num_sets1)
  )
)
```

Running the model

```
# Parameters to monitor
parameters <- c("sd_beta_s", "beta_sg")
```

```
jags_model <- jags.model(file="model.txt", data=data_jags1, n.chains=3, n.adapt=1000, inits = inits_list)
```

```
samples <- coda.samples(model=jags_model, variable.names=parameters, n.iter=5000)
```

Mixing and Convergence

```
mixing_plot <- function(chains = 1:3, parameter = 1, cols = 1:3, model = samples) {
  # This will now collect the data for the specified parameter across the specified chains.
  data_to_plot <- sapply(chains, function(i) model[[i]][, parameter])

  # Since `parameter` is an index, use it directly to get the name outside of sapply
  name = colnames(model[[1]])[parameter]

  # Plot using matplot
  return(matplot(data_to_plot, type = 'l', lty = 1,
    xlab = "Iteration", ylab = "Value",
    main = paste("Parameter ", name),
    col = cols)) # Assign different colors for each chain
}

# Setting graphical parameters to plot multiple plots on one graphical device
par(mfrow = c(1,2))

# Example call to the function to plot the second parameter
mixing_plot(parameter = 300, cols = c('black', 'darkred', 'darkblue'))
mixing_plot(parameter = 8390, cols = c('gray', 'indianred', 'lightblue'))
```

Gelman Plot


```

parameters_of_interest = c('beta_sg[999]', 'sd_beta_s[10]')

selected_samples <- lapply(samples, function(chain) {
  chain[, parameters_of_interest, drop = FALSE]
})

selected_samples <- as.mcmc.list(selected_samples)

gelman.plot(selected_samples)

```

gelman diagnostic.

Most of the sets are under the threshold. A few of them are way out of the threshold.

```

parameters_of_interest = c(8384:8452)

selected_samples <- lapply(samples, function(chain) {
  chain[, parameters_of_interest, drop = FALSE]
})

selected_samples <- as.mcmc.list(selected_samples)
diagnostic = gelman.diag(selected_samples)
diagnostic = as.data.frame(diagnostic$psrf)
diagnostic$set = paste('Set ', 1:69)

# Sorting the dataframe by 'Point est.'
diagnostic <- diagnostic[order(diagnostic$`Point est.`),]

# Update the factor levels after sorting
diagnostic$set <- factor(diagnostic$set, levels = unique(diagnostic$set))

# Setting up the plot area
plot(diagnostic$set, diagnostic$`Point est.` , type = 'n', # 'n' for no plotting
      xaxt = 'n',
      xlab = '', ylab = 'Gelman Point Estimate', pch = 19,
      ylim = range(c(1, 1.2)))

# Adding points for the point estimates
points(diagnostic$set, diagnostic$`Point est.` , pch = 19, col = "gray", cex = 1)

# Adding points for the Upper Confidence Interval
points(diagnostic$set, diagnostic$`Upper C.I.` , pch = 19, col = "darkred", cex = 1)

# Adding lollipop sticks - segments from point estimates to upper CI
segments(as.numeric(diagnostic$set), diagnostic$`Point est.` ,
         as.numeric(diagnostic$set), diagnostic$`Upper C.I.` , col = 'black')

# Adding a dashed line at y = 1.05
abline(h = 1.05, col = "blue", lty = "dashed")

# Custom x-axis with labels rotated 90 degrees
axis(1, at = 1:length(levels(diagnostic$set)), labels = levels(diagnostic$set), las = 2, cex = 0.8)

# Adding a legend to the plot
legend("topleft", legend = c("Point Estimate", "Upper C.I.", "Threshold (1.05)"),

```

```
col = c("gray", "darkred", "blue"), pch = c(19, 19, NA), lty = c(NA, NA, "dashed"))
```

Burning the first 2000 iterations

Summary for all the genes and gene sets. - This is the dataframe with the posteriors summary for beta_sg and beta_sd

```
jags_model <- jags.model(file="model.txt", data=data_jags1, n.chains=3, n.adapt=1000, inits = inits_list)

samples <- coda.samples(model=jags_model, variable.names=parameters, n.iter=10000)

runs = window(samples, 2001)

selected_samples1 = summary(runs)
selected_samplesq = as.data.frame(selected_samples1$quantiles)
selected_samplest = as.data.frame(selected_samples1$statistics)
```

Subsetting for genes

```
posterior_genes_sum = selected_samplest[grepl('beta_sg', rownames(selected_samplest)), ]
posterior_genes_quant = selected_samplesq[grepl('beta_sg', rownames(selected_samplesq)), ]

posterior_genes_all = cbind(posterior_genes_sum, posterior_genes_quant)
posterior_genes_all$genes = sets_unlist$genes

posterior_genes_all$status = ifelse(posterior_genes_all$`2.5%` < 0 & posterior_genes_all$`97.5%` < 0, 'L', 'H')

write.table(posterior_genes_all, 'Posterior_distributions_genes.txt', sep = ',')
```

Let's visualize the top 20 genes for each different group

```
posterior_genes_all |> group_by(status) |> top_n(20, wt = abs(Mean)) |>
  ggplot(aes(x = reorder(genes, Mean, decreasing = T), y = Mean)) +
  geom_hline(yintercept = 0, linetype = 'dashed') +
  geom_errorbar(aes(ymin = `2.5%`, ymax = `97.5%`), col = 'gray50') +
  geom_point(aes(col = status), show.legend = F) +
  theme_bw() +
  theme(axis.text.x = element_text(angle = 90, hjust = 1), plot.background = element_blank()) + xlab('')
```

Subsetting for sets

```
posterior_sets_sum = selected_samplest[grepl('sd_beta', rownames(selected_samplest)), ]
posterior_sets_quant = selected_samplesq[grepl('sd_beta', rownames(selected_samplesq)), ]

posterior_sets_all = cbind(posterior_sets_sum, posterior_sets_quant)
posterior_sets_all$sets = unique(sets_unlist$set)
posterior_sets_all$spread = posterior_sets_all$`97.5%` - posterior_sets_all$`2.5%`

write.table(posterior_sets_all, 'Posterior_distributions_sets.txt', sep = ',')
```

Let's visualize genes with high variance.

```
sets_ex = posterior_sets_all |> mutate(sets = gsub('HALLMARK_', '', sets)) |> filter(Mean > 10)
sets_ex = sets_ex[order(sets_ex$Mean, decreasing = T), ]
sets_ex$sets = factor(sets_ex$sets, levels = sets_ex$sets)

posterior_sets_all |> mutate(sets = gsub('HALLMARK_', '', sets)) |> filter(Mean > 10) |>
```

```
ggplot(aes(x = reorder(sets, Mean, decreasing = T), y = Mean)) +
  geom_errorbar(aes(ymin = `2.5%`, ymax = `97.5%`), col = 'gray50') +
  geom_point(col = 'red') +
  theme_bw() +
  theme(axis.text.x = element_text(angle = 90, hjust = 1), plot.background = element_blank()) + xlab('')
```

Analyzing data under the conventional method

Differential Expression using DESEQ2

```
### Differential Expression

### We use the count matrix that has the non normalized data

### Keeping rows with at least 10 counts (Removing genes with low counts)

metadata = data.frame(samples = colnames(count.matrix),
                      condition = rep(c('Control', 'Tumor'), each = 5))
metadata$condition = factor(metadata$condition, levels = c('Control', 'Tumor'))
rownames(metadata) = colnames(count.matrix)

dds = DESeqDataSetFromMatrix(countData = count.matrix,
                             colData = metadata,
                             design = ~condition)

dds = DESeq(dds)

results = results(dds) |> as.data.frame()
results = na.omit(results)
results$genes = rownames(results)

results$score = results$log2FoldChange * -log10(results$padj)
results2 = results |> dplyr::select(genes, score) |>
  na.omit() |>
  distinct() |> group_by(genes) |> top_n(1)
### use results2 for gsea

### let's filter only for the genes we analyzed
results = results |> filter(genes %in% posterior_genes_all$genes)
results$status = ifelse(results$padj < 0.05 & results$log2FoldChange < 0, 'Downregulated', ifelse(results$log2FoldChange > 0, 'Upregulated', 'Not significant'))

# Overlapping
## Upreg
bay_up = posterior_genes_all |> filter(status == 'Upregulated') |> select(genes) |> as.vector() |> unlist()

deseq_up = results |> filter(status == 'Upregulated') |> select(genes) |> as.vector() |> unlist()

x = list(`Bayesian Method` = bay_up, `Conventional Method` = deseq_up)

bay_up = posterior_genes_all |> filter(status == 'Downregulated') |> select(genes) |> as.vector() |> unlist()
```

```
deseq_up = results |> filter(status == 'Downregulated') |> select(genes) |> as.vector() |> unlist()

y = list(`Bayesian Method` = bay_up, `Conventional Method` = deseq_up)
```

Gene set enrichment using FGSEA

```
#### fgsea
```

```
### Sorting
```

```
ranks = deframe(results2)
ranks = sort(ranks, decreasing = T)
```

```
list_from_df = split(sets_unlist$genes, sets_unlist$set)
```

```
fgseares = fgseaMultilevel(pathways = list_from_df, stats = ranks)
```

```
fgseares = fgseares |> as.data.frame()
```

```
fgseares = fgseares |> mutate(pathway = gsub('HALLMARK_', '', pathway))
fgseares = fgseares |>
  filter(pathway %in% sets_ex$sets)
```

```
fgseares$pathway = factor(fgseares$pathway, levels = sets_ex$sets)
fgseares |>
```

```
ggplot(aes(pathway, abs(NES))) +
  geom_hline(yintercept = 0) +
  geom_segment(aes(y = 0, yend = abs(NES))) +
  geom_point(aes(fill = padj<0.05), show.legend = F, pch = 21, size = 3) +
  theme_bw() +
  theme(axis.text.x = element_text(angle = 90, hjust = 1), plot.background = element_blank()) + xlab('')
```